

Design and Deployment of a Highly Available, Multi-Tier Cloud-Native Application using the AWS Well-Architected Framework

A Case Study Report

Submitted by: Harshit Gupta

Roll Number: 2400290120111

KIET Group of Institutions

1. Introduction

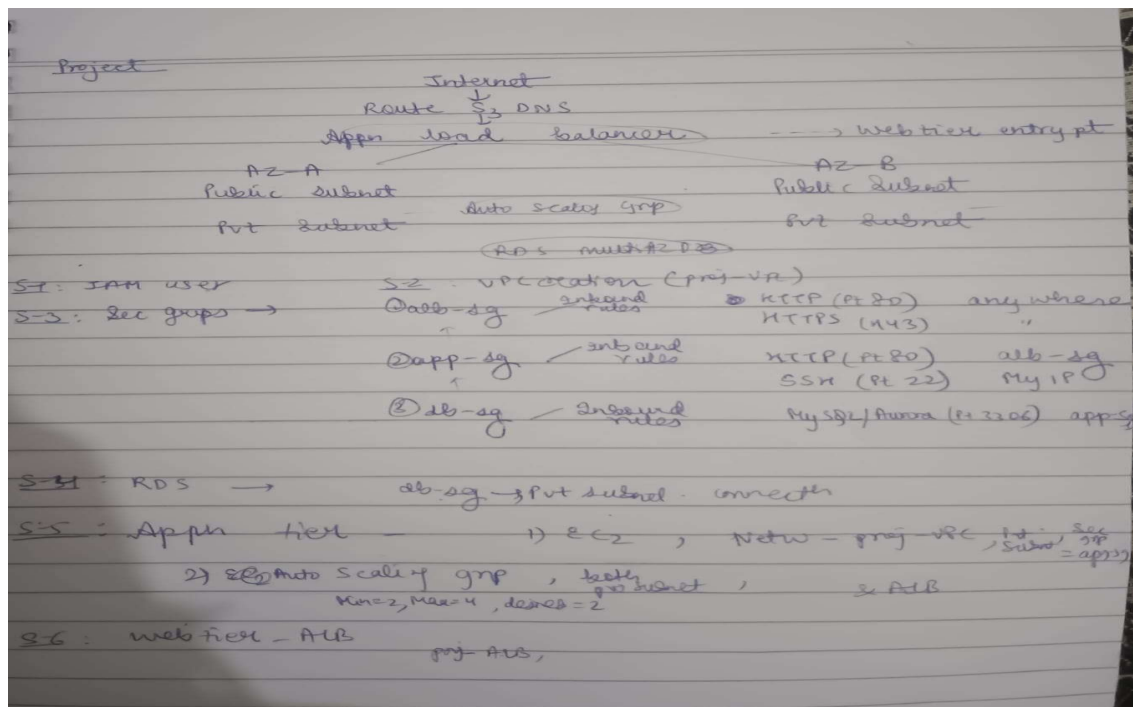
Modern applications need to remain available, reliable, and responsive even when individual components experience failures or sudden increases in user traffic. A highly available cloud architecture helps reduce downtime by distributing application resources across multiple Availability Zones and using services such as load balancing, auto scaling, and managed databases. Cloud-native design further improves scalability, flexibility, security, and operational efficiency by making use of managed AWS services rather than relying entirely on traditional single-server infrastructure.

This project, titled **“Design and Deployment of a Highly Available, Multi-Tier Cloud-Native Application using the AWS Well-Architected Framework,”** demonstrates the design and deployment of a web application using Amazon Web Services (AWS). The project focuses on implementing a multi-tier architecture with appropriate networking, compute, database, security, and monitoring components while following the principles of the AWS Well-Architected Framework. It was developed as part of a **college case study using AWS Free Tier/trial credits**, with cost considerations taken into account during resource selection and deployment. This report presents the architecture, implementation steps, security configuration, testing, monitoring, and overall evaluation of the proposed solution.

2. Objectives

- Design a fault-tolerant, multi-tier architecture using AWS managed services.
- Apply the AWS Well-Architected Framework across all six pillars.
- Demonstrate automatic recovery from instance and database failures.
- Operate within a constrained cloud budget through cost-conscious design choices.

3. Architecture Overview



3.1 Components

- Amazon VPC — custom network with public/private subnets across 2 AZs
- Application Load Balancer — distributes incoming traffic across app instances
- EC2 Auto Scaling Group — application tier, scales based on CPU utilization

- Amazon RDS (Multi-AZ) — managed relational database with automatic failover
- NAT Gateway — allows private subnet resources outbound internet access
- CloudWatch, CloudTrail, Secrets Manager — monitoring, auditing, and credential security

4. Implementation Steps

The implementation began with setting up the AWS account and applying basic cost and security controls. AWS Budgets and billing alerts were configured to monitor the project expenditure because the application was developed using AWS trial credits. An IAM user was created for regular AWS access, MFA was enabled, and the project was deployed in a single AWS region.

Next, a custom **VPC** named project-vpc was created using two Availability Zones, with two public subnets and two private subnets. An Internet Gateway was used for public connectivity and a single NAT Gateway was configured to provide outbound internet access to resources in the private subnets while reducing project cost.

Three tier-specific **Security Groups** were configured. The alb-sg allowed HTTP traffic from the internet, app-sg allowed application traffic only from the load balancer and restricted SSH access to the user's IP, while db-sg allowed database traffic only from the application servers. This ensured that the application and database tiers were not directly exposed to the internet.

For the database tier, **Amazon RDS** was provisioned using MySQL in the project's VPC. The database was configured in private subnets with public access disabled and the db-sg security group attached. Multi-AZ deployment was enabled to provide database high availability and automatic failover.

The application tier was then prepared using **Amazon EC2** with an Amazon Linux 2023 instance. The instance was placed in a private subnet and configured with the application environment and required startup commands. A **Launch Template** was created from the configured instance so that identical application instances could be launched automatically. An **Auto Scaling Group** was configured across both private subnets, with a desired and minimum capacity of two instances and a maximum capacity of four. Target-tracking scaling based on CPU utilization was also configured.

An internet-facing **Application Load Balancer (ALB)** was subsequently created in the two public subnets. The ALB used the alb-sg security group and forwarded HTTP requests on port 80 to the application target group. Health checks were configured so that unhealthy EC2 instances could be removed from traffic automatically. The ALB was connected to the Auto Scaling Group, allowing incoming requests to be distributed across application instances in different Availability Zones.

Finally, the application servers were connected to the **RDS database** using the RDS endpoint. Database connectivity was tested from the application tier, and the application configuration was updated with the database connection details. The Launch Template was updated so that newly created instances could use the same database configuration.

The deployment was then prepared for monitoring and validation using **Amazon CloudWatch**, including monitoring of EC2, ALB, and RDS metrics, while VPC Flow Logs and CloudTrail provided additional visibility and auditing. High availability was tested by stopping or terminating an application instance and observing the ALB health checks and Auto Scaling Group replacement process. The detailed commands, configuration values, and deployment settings are maintained separately in the **Deployment_Files/** folder and are referenced rather than repeated in this section.

5. Well-Architected Framework Mapping

The table below maps key design decisions to each of the six Well-Architected pillars.

Pillar	What We Did
Operational Excellence	Repeatable EC2 Launch Template; CloudWatch dashboards & alarms; CloudTrail auditing
Security	Least-privilege security groups per tier; private subnets for app/DB; Secrets Manager for credentials; IAM roles instead of hardcoded keys; MFA enabled

Reliability	Multi-AZ VPC design; Auto Scaling replaces failed instances automatically; RDS Multi-AZ automatic failover; ALB health checks
Performance Efficiency	Target-tracking auto scaling matches capacity to load; managed RDS removes DB operations overhead
Cost Optimization	Free-tier eligible instance sizes; single NAT Gateway; AWS Budgets/alarms; scale-to-zero when idle
Sustainability	Auto Scaling avoids over-provisioning idle capacity; managed services run on AWS's shared, efficient infrastructure

6. Testing & High-Availability Validation

To validate that the architecture is genuinely highly available (not just theoretically), the following failure scenarios were tested. Screenshots are provided in the `Deployment_Screenshots/` folder.

6.1 EC2 Instance Failure

To test the high-availability of the application tier, one running EC2 instance was manually terminated from the EC2 console. The Auto Scaling Group detected that the number of healthy instances had fallen below the configured desired capacity and automatically launched a replacement instance.

During the test, the application remained accessible through the **Application Load Balancer (ALB)** because the healthy instance in the other Availability Zone continued to serve incoming requests. After the replacement instance became healthy and passed the configured load balancer health checks, it was added back to the target group and became available to handle application traffic.

This test demonstrated that the combination of **Multi-AZ deployment, ALB health checks, and Auto Scaling** allowed the application to remain available even when an individual EC2 instance failed.

6.2 Database Failover

To test the high availability of the database tier, an **RDS “Reboot with failover”** operation was manually triggered. This simulated an Availability Zone failure by forcing the Amazon RDS primary database instance to fail over to its standby instance in the second Availability Zone.

During the failover, the application temporarily re-established its database connection. The standby database instance was promoted to the primary role, while the application continued using the same RDS endpoint without requiring any manual configuration changes. Once the failover was completed, normal database connectivity and application functionality were restored.

This test demonstrated the effectiveness of **RDS Multi-AZ deployment** in providing database availability and automatic failover in the event of a failure affecting the primary database instance.

6.3 Monitoring

Amazon CloudWatch was used to monitor the performance, availability, and health of the deployed application. A CloudWatch dashboard was configured to provide an overview of important metrics from the application infrastructure, including **EC2 CPU utilization, Application Load Balancer request count and latency, and RDS database connections and CPU utilization**.

CloudWatch alarms were also configured to identify important infrastructure conditions. An alarm was configured for an **unhealthy ALB target count greater than zero**, which could indicate that an application instance had become unavailable. Another alarm monitored **RDS CPU utilization above 80%**, helping identify unusually high database workload.

During the EC2 failure test, the ALB health-check metrics could be used to observe the affected instance becoming unhealthy while the remaining healthy instance continued serving requests. The Auto Scaling Group subsequently launched a replacement instance. The monitoring dashboard therefore provided visibility into the application's behavior during the failure and recovery process.

These monitoring and alerting mechanisms supported the **Operational Excellence** and **Reliability** pillars of the AWS Well-Architected Framework by providing continuous visibility into the health and performance of the deployed resources.

7. Cost Analysis

The project was deployed using the AWS Free Tier/trial account with a **\$100 trial credit**. According to the final AWS Cost Explorer/Billing dashboard, the total project expenditure was **\$10**, leaving approximately **\$90** of the available \$100 credit unused.

Several cost-optimization measures were followed throughout the implementation. Free-tier eligible and appropriately sized instance types were selected for the EC2 and RDS resources. A **single NAT Gateway** was used instead of one NAT Gateway per Availability Zone, reducing the hourly networking cost while remaining sufficient for this college demonstration project. Resources were also scaled down or stopped between work sessions so that they were not unnecessarily running 24/7. In addition, **AWS Budgets and billing alarms** were configured to proactively monitor expenditure and provide warnings before costs could become significant.

These decisions helped keep the project within the available trial credit while still demonstrating the key components of a highly available, multi-tier AWS architecture.

8. Challenges Faced

During the implementation, several configuration issues were encountered and resolved while connecting the different AWS services.

1. Private subnet connectivity:

The application EC2 instances were placed in private subnets as required by the architecture, which meant they could not be accessed directly from the internet. This initially made instance-level testing and troubleshooting difficult. The issue was addressed by using the appropriate private-subnet connectivity approach, including the EC2 Instance Connect Endpoint, while keeping the application instances private.

2. Security Group configuration:

Communication between the ALB, EC2 application instances, and RDS database initially required careful configuration of the inbound rules. The security groups were configured according to the tiered architecture: the application tier accepts traffic from the ALB security group, while the database accepts traffic only from the application security group. This resolved connectivity issues while maintaining restricted access between the tiers.

3. RDS connectivity:

Connecting the application tier to the RDS database required verifying the RDS endpoint, database port, private-subnet configuration, and security group rules. Database connectivity was tested from the application instance and the configuration was updated to use the RDS endpoint. This allowed the application to communicate with the database without exposing RDS to the public internet.

These issues provided practical experience with AWS networking, security, and troubleshooting and helped demonstrate why correct configuration of each tier is important in a multi-tier cloud architecture.

9. Conclusion

This project successfully demonstrated the design and deployment of a **highly available, multi-tier cloud-native application on AWS**. The application was designed across two Availability Zones using public and private subnets, an Application Load Balancer, an Auto Scaling Group with EC2 instances, and an Amazon RDS Multi-AZ database. Failure testing demonstrated that the application could continue serving requests when an EC2 instance was terminated, while the Auto Scaling Group automatically launched a replacement. Database failover testing also demonstrated the high-availability capability of the RDS layer.

Through this project, we gained practical understanding of AWS networking, compute, load balancing, auto scaling, managed databases, security groups, monitoring, and cost management. More importantly, the project showed how the **AWS Well-Architected Framework** can be applied in practice through its six pillars: Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, and Sustainability.

As future improvements, the architecture could be enhanced by adding **HTTPS using AWS Certificate Manager, AWS WAF for additional application-layer protection, and Infrastructure-as-Code using AWS CloudFormation or AWS CDK**. These improvements would make the deployment more secure, repeatable, and closer to a production-grade cloud environment.

•